

CSE 216: Database Sessional

PL/SQL Past Examination Question Papers & Complete Solutions

A Comprehensive, Chronologically Sorted Master Archive (2017 – 2023)

Clean Oracle PL/SQL Solutions with Error-Free Implementations, Triggers, Functions, Procedures, and Cursors

Document Coverage & Topics Included:

- **17 Batch:** 17-A1+B2, 17-A2, 17-B1 (Procedures, Functions, Triggers, Exceptions)
- **18 Batch:** 18 Comprehensive (Options, Limits, Dept Allocation & Trigger Audit)
- **19 Batch:** 19-A (Trigger), 19-A, 19-B (DML Triggers, Building/Salary Rules)
- **20 Batch:** 20-A1 (Trigger), 20-A1, 20-A2+B2 (Trigger), 20-A2+B2, 20-B1 (Trigger), 20-B1 (Academic Schema: Prereq, Takes, CGPA; HR Schema Triggers)
- **21 Batch:** 21-A, 21-B, 21-C, 21-Practice (String Matching, LPET, Rankings, Mutating Trigger Solutions)
- **22 Batch:** 22-A, 22-B, 22-C (Multi-table Joins, Exchanges, Subordinate Counts, Demotion Triggers)
- **23 Batch:** 23-A, 23-B, 23-C (Job History Archive, Department Audit, 50k Cap Compound Trigger, Overpay Verification)

Contents

1	2017 Batch Questions & Solutions	2
1.1	17 – A1 + B2 Online	2
1.2	17 – A2 Online	5
1.3	17 – B1 Online	7
2	2018 Batch Questions & Solutions	9
2.1	18 Online (Department Allocation & Summary Trigger)	9
3	2019 Batch Questions & Solutions	11
3.1	19 – A (Trigger) Online	11
3.2	19 – A Online	13
3.3	19 – B Online	15
4	2020 Batch Questions & Solutions	17
4.1	20 – A1 (Trigger) Online	17
4.2	20 – A1 Online (Academic Schema)	19
4.3	20 – A2 + B2 (Trigger) Online	21
4.4	20 – A2 + B2 Online	23
4.5	20 – B1 (Trigger) Online	25
4.6	20 – B1 Online	26
5	2021 Batch Questions & Solutions	28
5.1	21 – A Online	28
5.2	21 – B Online	30
5.3	21 – C Online	32
5.4	21 – Practice	34
6	2022 Batch Questions & Solutions	36
6.1	22 – A Online	36
6.2	22 – B Online	38
6.3	22 – C Online	41
7	2023 Batch Questions & Solutions	43
7.1	23 – A Online	43
7.2	23 – B Online	45
7.3	23 – C Online	47

1 2017 Batch Questions & Solutions

1.1 17 – A1 + B2 Online

17-A1+B2 Question 1: Salary Increment for Senior Managers

Write a PL/SQL block that increases the salary (by 15%) of each manager in the Employee table who has worked for 24 years or more. Output the last name, department id, old salary, and new salary for the managers whose salary was updated. Also, output the total number of updates in the table.

```

1 SET SERVEROUTPUT ON;
2 DECLARE
3     v_total_updates NUMBER := 0;
4     CURSOR c_mgrs IS
5         SELECT DISTINCT m.employee_id, m.last_name, m.department_id, m.salary
6         FROM employees m
7         JOIN employees e ON m.employee_id = e.manager_id
8         WHERE MONTHS_BETWEEN(SYSDATE, m.hire_date) / 12 >= 24;
9 BEGIN
10     FOR r IN c_mgrs LOOP
11         UPDATE employees
12         SET salary = salary * 1.15
13         WHERE employee_id = r.employee_id;
14
15         DBMS_OUTPUT.PUT_LINE('Last Name: ' || r.last_name ||
16                               ' | Dept ID: ' || NVL(TO_CHAR(r.department_id), 'N/A') ||
17                               ' | Old Sal: ' || r.salary ||
18                               ' | New Sal: ' || (r.salary * 1.15));
19         v_total_updates := v_total_updates + 1;
20     END LOOP;
21     DBMS_OUTPUT.PUT_LINE('Total Updates: ' || v_total_updates);
22     COMMIT;
23 END;
24 /

```

17-A1+B2 Question 2: Interchange Salary Procedure

Write a PL/SQL procedure called INTERCHANGE_SALARY that takes two employee id EID1 and EID2 as input and interchanges the salary of these two employees. Print previous and new salary of these two employees. Be sure to handle appropriate exceptions. Finally, write a PL/SQL block to call the procedure.

```

1 CREATE OR REPLACE PROCEDURE INTERCHANGE_SALARY(EID1 IN NUMBER, EID2 IN NUMBER) IS
2     v_sal1 employees.salary%TYPE;
3     v_sal2 employees.salary%TYPE;
4 BEGIN
5     SELECT salary INTO v_sal1 FROM employees WHERE employee_id = EID1;
6     SELECT salary INTO v_sal2 FROM employees WHERE employee_id = EID2;
7
8     DBMS_OUTPUT.PUT_LINE('Before Swap: Emp ' || EID1 || ' = ' || v_sal1 || ', Emp ' ||
9                           EID2 || ' = ' || v_sal2);
10
11     UPDATE employees SET salary = v_sal2 WHERE employee_id = EID1;
12     UPDATE employees SET salary = v_sal1 WHERE employee_id = EID2;
13
14     DBMS_OUTPUT.PUT_LINE('After Swap: Emp ' || EID1 || ' = ' || v_sal2 || ', Emp ' ||
15                           EID2 || ' = ' || v_sal1);
16     COMMIT;
17 EXCEPTION
18     WHEN NO_DATA_FOUND THEN
19         DBMS_OUTPUT.PUT_LINE('Error: One or both employee IDs do not exist.');
```

```

18     WHEN OTHERS THEN
19         DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);
20 END INTERCHANGE_SALARY;
21 /
22
23 -- Calling block
24 BEGIN
25     INTERCHANGE_SALARY(101, 102);
26 END;
27 /

```

17-A1+B2 Question 3: Long-Serving Employees in Accounting

Write a PL/SQL function which finds the employees working in the ACCOUNTING department for more than 20 years, and outputs a message for each of them. The number of employees who fulfill the criteria should be counted and returned. Then, write a PL/SQL block to call the function and output the count.

```

1 CREATE OR REPLACE FUNCTION COUNT_LONG_SERVING_ACCOUNTING RETURN NUMBER IS
2     v_count NUMBER := 0;
3     CURSOR c_emp IS
4         SELECT e.first_name, e.last_name, e.hire_date
5         FROM employees e
6         JOIN departments d ON e.department_id = d.department_id
7         WHERE UPPER(d.department_name) = 'ACCOUNTING'
8         AND MONTHS_BETWEEN(SYSDATE, e.hire_date) / 12 > 20;
9 BEGIN
10     FOR r IN c_emp LOOP
11         DBMS_OUTPUT.PUT_LINE('Eligible Employee: ' || r.first_name || ' ' ||
12                               r.last_name ||
13                               ' (Hired: ' || r.hire_date || ')');
14         v_count := v_count + 1;
15     END LOOP;
16     RETURN v_count;
17 EXCEPTION
18     WHEN OTHERS THEN
19         DBMS_OUTPUT.PUT_LINE('Function error: ' || SQLERRM);
20         RETURN 0;
21 END COUNT_LONG_SERVING_ACCOUNTING;
22 /
23
24 -- Calling block
25 DECLARE
26     v_total NUMBER;
27 BEGIN
28     v_total := COUNT_LONG_SERVING_ACCOUNTING();
29     DBMS_OUTPUT.PUT_LINE('Total Accounting employees serving > 20 years: ' || v_total);
30 END;
31 /

```

17-A1+B2 Question 4: Prevent Salary Decrease Trigger

Copy the contents of the employee table to create a new table titled Employee_2. Create a Trigger called NO_DEC to ensure salary is not decreased for any employees in the Employee_2 table.

```

1 -- Setup table
2 CREATE TABLE Employee_2 AS SELECT * FROM employees;
3
4 CREATE OR REPLACE TRIGGER NO_DEC
5 BEFORE UPDATE OF salary ON Employee_2
6 FOR EACH ROW
7 BEGIN

```

```
8      IF :NEW.salary < :OLD.salary THEN
9          RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be decreased for employee ID '
10             || :OLD.employee_id);
11      END IF;
12  END;
13  /
```

1.2 17 – A2 Online

17-A2: Default Commission Percentage & Mass Update

Table setup: EMP_TEMP(EMPLOYEE_ID, NAME, SALARY, COMMISSION_PCT, MANAGER_ID, DEPARTMENT_ID). 1. Implement function defaultCommissionPCT(p_mgr_id): - If manager_id >= 123: 10% of avg commission_pct of employees under this manager (treat nulls as 0). - If manager_id between 110 and 122: 12% of avg commission_pct of employees under this manager. - Otherwise: 0.1 fixed. 2. Implement procedure changeAll to inspect EMP_TEMP. If commission_pct is null, set it using the function; if valid, do nothing. Print messages indicating status.

```

1 CREATE OR REPLACE FUNCTION defaultCommissionPCT(p_mgr_id IN NUMBER) RETURN NUMBER IS
2     v_avg_comm NUMBER := 0;
3     v_result NUMBER := 0.1;
4 BEGIN
5     IF p_mgr_id IS NULL THEN
6         RETURN 0.1;
7     END IF;
8
9     SELECT NVL(AVG(NVL(commission_pct, 0)), 0)
10    INTO v_avg_comm
11   FROM EMP_TEMP
12  WHERE manager_id = p_mgr_id;
13
14     IF p_mgr_id >= 123 THEN
15         v_result := 0.10 * v_avg_comm;
16     ELSIF p_mgr_id BETWEEN 110 AND 122 THEN
17         v_result := 0.12 * v_avg_comm;
18     ELSE
19         v_result := 0.1;
20     END IF;
21
22     RETURN v_result;
23 EXCEPTION
24     WHEN OTHERS THEN
25         RETURN 0.1;
26 END defaultCommissionPCT;
27 /
28
29 CREATE OR REPLACE PROCEDURE changeAll IS
30     v_def NUMBER;
31     CURSOR c_emp IS SELECT employee_id, manager_id, commission_pct FROM EMP_TEMP FOR
32                       UPDATE OF commission_pct;
33 BEGIN
34     FOR r IN c_emp LOOP
35         IF r.commission_pct IS NULL THEN
36             v_def := defaultCommissionPCT(r.manager_id);
37             UPDATE EMP_TEMP
38             SET commission_pct = v_def
39             WHERE CURRENT OF c_emp;
40             DBMS_OUTPUT.PUT_LINE('Emp ' || r.employee_id || ': Commission updated to '
41                                   || v_def);
42         ELSE
43             DBMS_OUTPUT.PUT_LINE('Emp ' || r.employee_id || ': Commission already
44                                   valid (' || r.commission_pct || ')');
45         END IF;
46     END LOOP;
47     COMMIT;
48 EXCEPTION
49     WHEN OTHERS THEN
50         DBMS_OUTPUT.PUT_LINE('Error in changeAll: ' || SQLERRM);
51 END changeAll;
52 /

```


1.3 17 – B1 Online

17-B1 Question 1: Manager Salary Ranking Procedure

Write a procedure taking 3 inputs (n employee count, m average salary, m_id manager ID) and returning r (rank of manager m_id). Finds managers managing $> n$ employees whose subordinates have average salary $< m$. Rank them ascending by average salary (least avg salary = Rank 1). Print managers' names, ranks, and avg salaries.

```

1 CREATE OR REPLACE PROCEDURE RANK MANAGERS (
2     n IN NUMBER,
3     m IN NUMBER,
4     m_id IN NUMBER,
5     r OUT NUMBER
6 ) IS
7     v_rank NUMBER := 0;
8     v_found BOOLEAN := FALSE;
9     CURSOR c_mgrs IS
10        SELECT e.manager_id, m.first_name || ' ' || m.last_name AS mgr_name,
11               AVG(e.salary) AS avg_sal
12        FROM employees e
13        JOIN employees m ON e.manager_id = m.employee_id
14        GROUP BY e.manager_id, m.first_name, m.last_name
15        HAVING COUNT(e.employee_id) > n AND AVG(e.salary) < m
16        ORDER BY AVG(e.salary) ASC;
17 BEGIN
18     r := NULL;
19     FOR rec IN c_mgrs LOOP
20         v_rank := v_rank + 1;
21         DBMS_OUTPUT.PUT_LINE('Rank: ' || v_rank || ' | Manager: ' || rec.mgr_name || '
22                               | Avg Sal: ' || ROUND(rec.avg_sal, 2));
23         IF rec.manager_id = m_id THEN
24             r := v_rank;
25             v_found := TRUE;
26         END IF;
27     END LOOP;
28     IF NOT v_found THEN
29         DBMS_OUTPUT.PUT_LINE('Manager ID ' || m_id || ' did not meet the criteria or
30                               does not exist.');
```

17-B1 Question 2: Employee Salary Rank Under Manager

Write a function that takes two parameters `employee_id` and `manager_id`, and returns the salary rank of the employee working under the manager (highest salary == rank 1).

```

1 CREATE OR REPLACE FUNCTION GET_EMP_SALARY_RANK (
2     p_emp_id IN NUMBER,
3     p_mgr_id IN NUMBER
4 ) RETURN NUMBER IS
5     v_rank NUMBER;
6 BEGIN
7     SELECT rn INTO v_rank
8     FROM (
9         SELECT employee_id, DENSE_RANK() OVER (ORDER BY salary DESC) AS rn
10        FROM employees
11        WHERE manager_id = p_mgr_id
12     )
13     WHERE employee_id = p_emp_id;
14
```

```
15     RETURN v_rank;  
16 EXCEPTION  
17     WHEN NO_DATA_FOUND THEN  
18         RETURN NULL;  
19     WHEN OTHERS THEN  
20         RETURN NULL;  
21 END GET_EMP_SALARY_RANK;  
22 /
```

2 2018 Batch Questions & Solutions

2.1 18 Online (Department Allocation & Summary Trigger)

18 Online: Department Assignment and Total Salary Tracking Trigger

Employees have department choices stored in `OPTIONS(employee_id, department_id, choice_order)`. Departments have capacities stored in `LIMITS(department_id, max_limit)`. If no limit, capacity is unlimited. Assign employees in ascending order of `hire_date` into `EMPLOYEE_DEPARTMENT(employee_id, department_id)`. Also, maintain department total salary in `DEPARTMENT_SALARY(department_id, total_salary)` via a trigger on `EMPLOYEE_DEPARTMENT`.

```

1  -- Schema Setup
2  CREATE TABLE OPTIONS (
3      employee_id NUMBER,
4      department_id NUMBER,
5      choice_order NUMBER
6  );
7
8  CREATE TABLE LIMITS (
9      department_id NUMBER PRIMARY KEY,
10     max_limit NUMBER
11 );
12
13 CREATE TABLE EMPLOYEE_DEPARTMENT (
14     employee_id NUMBER PRIMARY KEY,
15     department_id NUMBER
16 );
17
18 CREATE TABLE DEPARTMENT_SALARY (
19     department_id NUMBER PRIMARY KEY,
20     total_salary NUMBER DEFAULT 0
21 );
22
23 -- Trigger to maintain DEPARTMENT_SALARY
24 CREATE OR REPLACE TRIGGER TRG_EMPLOYEE_DEPT_SALARY
25 AFTER INSERT OR UPDATE OR DELETE ON EMPLOYEE_DEPARTMENT
26 FOR EACH ROW
27 DECLARE
28     v_old_sal NUMBER := 0;
29     v_new_sal NUMBER := 0;
30 BEGIN
31     IF DELETING OR UPDATING THEN
32         BEGIN
33             SELECT salary INTO v_old_sal FROM employees WHERE employee_id =
34                 :OLD.employee_id;
35             EXCEPTION WHEN NO_DATA_FOUND THEN v_old_sal := 0;
36             END;
37             UPDATE DEPARTMENT_SALARY
38             SET total_salary = total_salary - v_old_sal
39             WHERE department_id = :OLD.department_id;
40         END IF;
41
42     IF INSERTING OR UPDATING THEN
43         BEGIN
44             SELECT salary INTO v_new_sal FROM employees WHERE employee_id =
45                 :NEW.employee_id;
46             EXCEPTION WHEN NO_DATA_FOUND THEN v_new_sal := 0;
47             END;
48             MERGE INTO DEPARTMENT_SALARY ds
49             USING (SELECT :NEW.department_id AS did FROM dual) src
50             ON (ds.department_id = src.did)
51             WHEN MATCHED THEN
52                 UPDATE SET total_salary = total_salary + v_new_sal

```

```

51         WHEN NOT MATCHED THEN
52             INSERT (department_id, total_salary) VALUES (src.did, v_new_sal);
53     END IF;
54 END;
55 /
56
57 -- Stored Procedure for Assignment
58 CREATE OR REPLACE PROCEDURE ASSIGN_EMPLOYEES_TO_DEPTS IS
59     v_limit NUMBER;
60     v_current_count NUMBER;
61     v_assigned BOOLEAN;
62
63     CURSOR c_emp IS
64         SELECT employee_id, hire_date
65         FROM employees
66         ORDER BY hire_date ASC, employee_id ASC;
67
68     CURSOR c_opts(p_eid NUMBER) IS
69         SELECT department_id
70         FROM OPTIONS
71         WHERE employee_id = p_eid
72         ORDER BY choice_order ASC;
73 BEGIN
74     EXECUTE IMMEDIATE 'TRUNCATE TABLE EMPLOYEE_DEPARTMENT';
75     EXECUTE IMMEDIATE 'TRUNCATE TABLE DEPARTMENT_SALARY';
76
77     FOR emp IN c_emp LOOP
78         v_assigned := FALSE;
79         FOR opt IN c_opts(emp.employee_id) LOOP
80             BEGIN
81                 SELECT max_limit INTO v_limit FROM LIMITS WHERE department_id =
82                     opt.department_id;
83                 EXCEPTION WHEN NO_DATA_FOUND THEN
84                     v_limit := NULL; -- No limit
85             END;
86
87             SELECT COUNT(*) INTO v_current_count
88             FROM EMPLOYEE_DEPARTMENT
89             WHERE department_id = opt.department_id;
90
91             IF v_limit IS NULL OR v_current_count < v_limit THEN
92                 INSERT INTO EMPLOYEE_DEPARTMENT(employee_id, department_id)
93                 VALUES (emp.employee_id, opt.department_id);
94                 v_assigned := TRUE;
95                 EXIT; -- Assigned to highest available choice
96             END IF;
97         END LOOP;
98
99         IF NOT v_assigned THEN
100             DBMS_OUTPUT.PUT_LINE('Employee ' || emp.employee_id || ' could not be
101                 assigned. ');
102         END IF;
103     END LOOP;
104     COMMIT;
105 END ASSIGN_EMPLOYEES_TO_DEPTS;
106 /

```

3 2019 Batch Questions & Solutions

3.1 19 – A (Trigger) Online

19-A (Trigger) Setup & Questions 1, 2, 3

Table: emp_tmp(E_ID VARCHAR2(10) PK, NAME, AGE, SALARY, ADDR_LINE1, ADDR_LINE2, CITY, DISTRICT). **Q1:** Trigger executed BEFORE INSERT or DELETE displaying record details. **Q2:** Trigger AFTER UPDATE of SALARY (where new salary > 0 and < 999999), displaying old and new salary. **Q3:** Trigger AFTER DELETE on emp_tmp. If age < 30, insert into emp_young; else insert into emp_adult.

```

1  -- Q1: tr_insert_delete
2  CREATE OR REPLACE TRIGGER tr_insert_delete_student
3  BEFORE INSERT OR DELETE ON emp_tmp
4  FOR EACH ROW
5  BEGIN
6      IF INSERTING THEN
7          DBMS_OUTPUT.PUT_LINE('The inserted record is:');
8          DBMS_OUTPUT.PUT_LINE('EID = ' || :NEW.E_ID);
9          DBMS_OUTPUT.PUT_LINE('Name = ' || :NEW.NAME);
10         DBMS_OUTPUT.PUT_LINE('Salary = ' || :NEW.SALARY);
11     ELSIF DELETING THEN
12         DBMS_OUTPUT.PUT_LINE('The deleted record is:');
13         DBMS_OUTPUT.PUT_LINE('EID = ' || :OLD.E_ID);
14         DBMS_OUTPUT.PUT_LINE('Name = ' || :OLD.NAME);
15         DBMS_OUTPUT.PUT_LINE('Salary = ' || :OLD.SALARY);
16     END IF;
17 END;
18 /
19
20 -- Q2: tr_update
21 CREATE OR REPLACE TRIGGER tr_update_student
22 AFTER UPDATE OF SALARY ON emp_tmp
23 FOR EACH ROW
24 WHEN (NEW.SALARY > 0 AND NEW.SALARY < 999999)
25 BEGIN
26     DBMS_OUTPUT.PUT_LINE('The update information is:');
27     DBMS_OUTPUT.PUT_LINE('EID: ' || :NEW.E_ID);
28     DBMS_OUTPUT.PUT_LINE('Name: ' || :NEW.NAME);
29     DBMS_OUTPUT.PUT_LINE('SALARY before update: ' || :OLD.SALARY);
30     DBMS_OUTPUT.PUT_LINE('SALARY after update: ' || :NEW.SALARY);
31 END;
32 /
33
34 -- Q3: tr_backup
35 CREATE TABLE emp_young AS SELECT * FROM emp_tmp WHERE 1=2;
36 CREATE TABLE emp_adult AS SELECT * FROM emp_tmp WHERE 1=2;
37
38 CREATE OR REPLACE TRIGGER tr_backup_student
39 AFTER DELETE ON emp_tmp
40 FOR EACH ROW
41 BEGIN
42     IF :OLD.AGE < 30 THEN
43         INSERT INTO emp_young (E_ID, NAME, AGE, SALARY, ADDR_LINE1, ADDR_LINE2, CITY,
44             DISTRICT)
45         VALUES (:OLD.E_ID, :OLD.NAME, :OLD.AGE, :OLD.SALARY, :OLD.ADDR_LINE1,
46             :OLD.ADDR_LINE2, :OLD.CITY, :OLD.DISTRICT);
47     ELSE
48         INSERT INTO emp_adult (E_ID, NAME, AGE, SALARY, ADDR_LINE1, ADDR_LINE2, CITY,
49             DISTRICT)
50         VALUES (:OLD.E_ID, :OLD.NAME, :OLD.AGE, :OLD.SALARY, :OLD.ADDR_LINE1,
51             :OLD.ADDR_LINE2, :OLD.CITY, :OLD.DISTRICT);

```

```
48     END IF;  
49 END;  
50 /
```

3.2 19 – A Online

19-A Questions 1, 2, 3: Salary Block, Dept Comparison, Dept Placement

Q1: Total salary before and after 20% increase. **Q2:** Procedure DEPT_COMPARISON(DEPT1, DEPT2, msg OUT VARCHAR2). **Q3:** Function dept_placement(dept_id, val) returning building string.

```

1  -- Q1: PL/SQL Block
2  DECLARE
3      v_total_before NUMBER;
4      v_total_after  NUMBER;
5  BEGIN
6      SELECT SUM(salary) INTO v_total_before FROM employees;
7      DBMS_OUTPUT.PUT_LINE('The total salary of all employees is ' || v_total_before);
8
9      UPDATE employees SET salary = salary * 1.20;
10
11     SELECT SUM(salary) INTO v_total_after FROM employees;
12     DBMS_OUTPUT.PUT_LINE('The total updated salary of employees is ' || v_total_after);
13     ROLLBACK; -- rollback test change
14 END;
15 /
16
17 -- Q2: DEPT_COMPARISON
18 CREATE OR REPLACE PROCEDURE DEPT_COMPARISON(
19     DEPT1 IN NUMBER,
20     DEPT2 IN NUMBER,
21     p_msg OUT VARCHAR2
22 ) IS
23     c1 NUMBER := 0;
24     c2 NUMBER := 0;
25 BEGIN
26     SELECT COUNT(*) INTO c1 FROM employees WHERE department_id = DEPT1;
27     SELECT COUNT(*) INTO c2 FROM employees WHERE department_id = DEPT2;
28
29     IF c1 > c2 THEN
30         p_msg := 'DEPT 1 has higher number of employees than DEPT 2';
31     ELSIF c1 < c2 THEN
32         p_msg := 'DEPT 1 has lower number of employees than DEPT 2';
33     ELSE
34         p_msg := 'DEPT 2 has equal number of employees of DEPT 1';
35     END IF;
36     DBMS_OUTPUT.PUT_LINE(p_msg);
37 END DEPT_COMPARISON;
38 /
39
40 -- Q3: dept_placement
41 CREATE OR REPLACE FUNCTION dept_placement(p_dept_id IN NUMBER, p_val IN NUMBER)
42 RETURN VARCHAR2 IS
43 BEGIN
44     IF p_dept_id IS NULL THEN
45         RETURN 'Registrar Building';
46     ELSIF p_dept_id > p_val THEN
47         RETURN 'ECE Bhaban';
48     ELSE
49         RETURN 'Old Campus';
50     END IF;
51 END dept_placement;
52 /
53
54 -- Testing Q3
55 BEGIN
56     FOR r IN (SELECT employee_id, first_name, last_name, department_id FROM employees
57               WHERE employee_id IN (178, 200, 201)) LOOP
58         DBMS_OUTPUT.PUT_LINE(r.first_name || ' ' || r.last_name || ': ' ||

```

```
        dept_placement(r.department_id, 15));  
58     END LOOP;  
59 END;  
60 /
```

3.3 19 – B Online

19-B Questions 1, 2, 3: Countries Block, Salary Comparison, Salary Check

Q1: Tuple count in COUNTRIES before and after inserting a tuple. **Q2:** Procedure SALARY_COMPARISON(EID1, EID2, msg OUT VARCHAR2). **Q3:** Function salary_Check(p_eid, p_val) returning 'High', 'Low', or 'Equal'.

```

1  -- Q1: PL/SQL Block
2  DECLARE
3      v_count NUMBER;
4  BEGIN
5      SELECT COUNT(*) INTO v_count FROM countries;
6      DBMS_OUTPUT.PUT_LINE('The number of tuples in countries table is ' || v_count);
7
8      INSERT INTO countries (country_id, country_name, region_id)
9      VALUES ('ZZ', 'Zootopia', 1);
10
11     SELECT COUNT(*) INTO v_count FROM countries;
12     DBMS_OUTPUT.PUT_LINE('The number of tuples in countries table after insertion is '
13     || v_count);
14     ROLLBACK;
15 END;
16 /
17
18 -- Q2: SALARY_COMPARISON
19 CREATE OR REPLACE PROCEDURE SALARY_COMPARISON(
20     EID1 IN NUMBER,
21     EID2 IN NUMBER,
22     p_msg OUT VARCHAR2
23 ) IS
24     s1 employees.salary%TYPE;
25     s2 employees.salary%TYPE;
26 BEGIN
27     SELECT salary INTO s1 FROM employees WHERE employee_id = EID1;
28     SELECT salary INTO s2 FROM employees WHERE employee_id = EID2;
29
30     IF s1 > s2 THEN
31         UPDATE employees SET salary = s1 WHERE employee_id = EID2;
32         p_msg := 'EID 2 is updated by EID 1';
33     ELSIF s1 < s2 THEN
34         p_msg := 'EID 1 earns lower than EID 2';
35     ELSE
36         p_msg := 'EID 2 earns equals to EID 1';
37     END IF;
38     DBMS_OUTPUT.PUT_LINE(p_msg);
39 EXCEPTION
40     WHEN NO_DATA_FOUND THEN
41         p_msg := 'One or both employee IDs do not exist.';
42         DBMS_OUTPUT.PUT_LINE(p_msg);
43 END SALARY_COMPARISON;
44 /
45
46 -- Q3: salary_Check
47 CREATE OR REPLACE FUNCTION salary_Check(p_eid IN NUMBER, p_val IN NUMBER) RETURN
48     VARCHAR2 IS
49     v_sal employees.salary%TYPE;
50 BEGIN
51     SELECT salary INTO v_sal FROM employees WHERE employee_id = p_eid;
52     IF v_sal > p_val THEN
53         RETURN 'High';
54     ELSIF v_sal < p_val THEN
55         RETURN 'Low';
56     ELSE
57         RETURN 'Equal';
58     END IF;
59 END salary_Check;

```

```
56     END IF;  
57 EXCEPTION  
58     WHEN NO_DATA_FOUND THEN  
59         RETURN 'Employee Not Found';  
60 END salary_Check;  
61 /
```

4 2020 Batch Questions & Solutions

4.1 20 – A1 (Trigger) Online

20-A1 (Trigger): EMP_SAL_LOG Triggers

Table: EMP_SAL_LOG(LOG_ID, EMPLOYEE_ID, OLD_SALARY, NEW_SALARY, MOD_DATE, USER_ID, USERNAME, STATUS). Sequence: CREATE SEQUENCE emp_sal_log_seq START WITH 1 INCREMENT BY 1;

1. Trigger on EMPLOYEES for INSERT or DELETE: status = 'APPROVED', MOD_DATE = current date, log_id sequential, irrelevant fields null.
2. Trigger on EMPLOYEES for UPDATE OF salary: Ensure update is NOT < 1 month ago and change is NOT > 20%. If violated, user informed, STATUS='DENIED', rollback/revert salary. Else STATUS='APPROVED'.

```

1 CREATE SEQUENCE emp_sal_log_seq START WITH 1 INCREMENT BY 1;
2
3 -- Trigger 1: Insertion or Deletion
4 CREATE OR REPLACE TRIGGER trg_sal_log_ins_del
5 AFTER INSERT OR DELETE ON employees
6 FOR EACH ROW
7 BEGIN
8     IF INSERTING THEN
9         INSERT INTO EMP_SAL_LOG (LOG_ID, EMPLOYEE_ID, OLD_SALARY, NEW_SALARY,
10            MOD_DATE, USER_ID, USERNAME, STATUS)
11            VALUES (emp_sal_log_seq.NEXTVAL, :NEW.employee_id, NULL, :NEW.salary, SYSDATE,
12                SYS_CONTEXT('USERENV', 'SESSION_USERID'), SYS_CONTEXT('USERENV',
13                    'SESSION_USER'), 'APPROVED');
14     ELSIF DELETING THEN
15         INSERT INTO EMP_SAL_LOG (LOG_ID, EMPLOYEE_ID, OLD_SALARY, NEW_SALARY,
16            MOD_DATE, USER_ID, USERNAME, STATUS)
17            VALUES (emp_sal_log_seq.NEXTVAL, :OLD.employee_id, :OLD.salary, NULL, SYSDATE,
18                SYS_CONTEXT('USERENV', 'SESSION_USERID'), SYS_CONTEXT('USERENV',
19                    'SESSION_USER'), 'APPROVED');
20     END IF;
21 END;
22 /
23
24 -- Trigger 2: Salary Update Validation
25 CREATE OR REPLACE TRIGGER trg_sal_log_update
26 BEFORE UPDATE OF salary ON employees
27 FOR EACH ROW
28 DECLARE
29     PRAGMA AUTONOMOUS_TRANSACTION;
30     v_last_mod DATE;
31     v_pct_diff NUMBER;
32     v_is_denied BOOLEAN := FALSE;
33 BEGIN
34     -- Compute percent change
35     IF :OLD.salary > 0 THEN
36         v_pct_diff := ABS(:NEW.salary - :OLD.salary) / :OLD.salary;
37     ELSE
38         v_pct_diff := 0;
39     END IF;
40
41     -- Check last update date from log
42     BEGIN
43         SELECT MAX(MOD_DATE) INTO v_last_mod
44         FROM EMP_SAL_LOG
45         WHERE EMPLOYEE_ID = :OLD.employee_id AND STATUS = 'APPROVED';
46     EXCEPTION WHEN NO_DATA_FOUND THEN
47         v_last_mod := NULL;
48     END;

```

```
45
46 IF v_pct_diff > 0.20 THEN
47     v_is_denied := TRUE;
48 ELSIF v_last_mod IS NOT NULL AND MONTHS_BETWEEN(SYSDATE, v_last_mod) < 1 THEN
49     v_is_denied := TRUE;
50 END IF;
51
52 IF v_is_denied THEN
53     INSERT INTO EMP_SAL_LOG (LOG_ID, EMPLOYEE_ID, OLD_SALARY, NEW_SALARY,
54         MOD_DATE, USER_ID, USERNAME, STATUS)
55     VALUES (emp_sal_log_seq.NEXTVAL, :OLD.employee_id, :OLD.salary, :NEW.salary,
56         SYSDATE,
57         SYS_CONTEXT('USERENV', 'SESSION_USERID'), SYS_CONTEXT('USERENV',
58             'SESSION_USER'), 'DENIED');
59     COMMIT;
60     DBMS_OUTPUT.PUT_LINE('SALARY UPDATE FAILED');
61     RAISE_APPLICATION_ERROR(-20002, 'SALARY UPDATE FAILED: Restriction violated.');
```

```
59 ELSE
60     INSERT INTO EMP_SAL_LOG (LOG_ID, EMPLOYEE_ID, OLD_SALARY, NEW_SALARY,
61         MOD_DATE, USER_ID, USERNAME, STATUS)
62     VALUES (emp_sal_log_seq.NEXTVAL, :OLD.employee_id, :OLD.salary, :NEW.salary,
63         SYSDATE,
64         SYS_CONTEXT('USERENV', 'SESSION_USERID'), SYS_CONTEXT('USERENV',
65             'SESSION_USER'), 'APPROVED');
66     COMMIT;
67 END IF;
68 END;
```

```
65
66 /
```

4.2 20 – A1 Online (Academic Schema)

20-A1 Question 1: Course Prerequisites Comparison

Tables: PREREQ, COURSE, DEPARTMENT.

- Function taking two course IDs returning TRUE if course 1 is a prereq of strictly more courses than course 2, FALSE otherwise.
- Anonymous block: For all courses in 'Lambeau' building, compare prereq count against courses in 'Bronfman' building. Order by title. Cannot use INTO or WHILE.

```

1  -- 1a: Function
2  CREATE OR REPLACE FUNCTION HAS_MORE_PREREQS(c1 IN VARCHAR2, c2 IN VARCHAR2) RETURN
   BOOLEAN IS
3      cnt1 NUMBER := 0;
4      cnt2 NUMBER := 0;
5  BEGIN
6      SELECT COUNT(*) INTO cnt1 FROM prereq WHERE prereq_id = c1;
7      SELECT COUNT(*) INTO cnt2 FROM prereq WHERE prereq_id = c2;
8      RETURN (cnt1 > cnt2);
9  END HAS_MORE_PREREQS;
10 /
11
12 -- 1b: Anonymous Block (without INTO or WHILE)
13 BEGIN
14     FOR r1 IN (
15         SELECT c.course_id, c.title, (SELECT COUNT(*) FROM prereq p WHERE p.course_id
16             = c.course_id) AS pre_cnt
17         FROM course c JOIN department d ON c.dept_name = d.dept_name
18         WHERE d.building = 'Lambeau'
19         ORDER BY c.title ASC
20     ) LOOP
21         FOR r2 IN (
22             SELECT c.course_id, c.title, (SELECT COUNT(*) FROM prereq p WHERE
23                 p.course_id = c.course_id) AS pre_cnt
24             FROM course c JOIN department d ON c.dept_name = d.dept_name
25             WHERE d.building = 'Bronfman'
26             ORDER BY c.title ASC
27         ) LOOP
28             IF r1.pre_cnt > r2.pre_cnt THEN
29                 DBMS_OUTPUT.PUT_LINE(r1.title || ' needs more prerequisites than ' ||
30                     r2.title);
31             END IF;
32         END LOOP;
33     END LOOP;
34 END;
35 /

```

20-A1 Question 2: CGPA Trend Classification

Write a PL/SQL procedure taking Student ID as input and calculating CGPA per semester chronologically. If CGPA is monotonically non-decreasing, print CGPA is always increasing. If CGPA is strictly decreasing, print CGPA is always decreasing. Otherwise print CGPA is following a zig-zag pattern.

```

1  CREATE OR REPLACE PROCEDURE CHECK_CGPA_TREND(p_sid IN NUMBER) IS
2      v_prev_cgpa FLOAT := NULL;
3      v_is_inc BOOLEAN := TRUE;
4      v_is_dec BOOLEAN := TRUE;
5      v_count NUMBER := 0;
6
7      CURSOR c_sem IS
8          SELECT year, semester,

```

```

9          CASE semester WHEN 'Spring' THEN 1 WHEN 'Summer' THEN 2 WHEN 'Fall'
10             THEN 3 ELSE 4 END as sem_order
11 FROM takes
12 WHERE id = TO_CHAR(p_sid)
13 GROUP BY year, semester
14 ORDER BY year ASC, sem_order ASC;
15
16 v_cum_gp FLOAT;
17 v_cum_cr FLOAT;
18 v_curr_cgpa FLOAT;
19 BEGIN
20   FOR sem IN c_sem LOOP
21     v_count := v_count + 1;
22     -- Cumulative CGPA up to this semester
23     SELECT NVL(SUM(gp), 0), NVL(SUM(credits), 0)
24     INTO v_cum_gp, v_cum_cr
25     FROM (
26       SELECT MAX(GRADE_TO_POINT(t.grade) * c.credits) AS gp, c.credits
27       FROM takes t
28       JOIN course c ON t.course_id = c.course_id
29       WHERE t.id = TO_CHAR(p_sid)
30       AND (t.year < sem.year OR (t.year = sem.year AND
31         CASE t.semester WHEN 'Spring' THEN 1 WHEN 'Summer' THEN 2 WHEN
32            'Fall' THEN 3 ELSE 4 END <= sem.sem_order))
33       GROUP BY t.course_id, c.credits
34     );
35
36   IF v_cum_cr > 0 THEN
37     v_curr_cgpa := ROUND(v_cum_gp / v_cum_cr, 4);
38   ELSE
39     v_curr_cgpa := 0;
40   END IF;
41
42   IF v_prev_cgpa IS NOT NULL THEN
43     IF v_curr_cgpa < v_prev_cgpa THEN
44       v_is_inc := FALSE;
45     END IF;
46     IF v_curr_cgpa >= v_prev_cgpa THEN
47       v_is_dec := FALSE;
48     END IF;
49   END IF;
50   v_prev_cgpa := v_curr_cgpa;
51 END LOOP;
52
53 IF v_count <= 1 THEN
54   DBMS_OUTPUT.PUT_LINE('CGPA is always increasing');
55 ELSIF v_is_inc THEN
56   DBMS_OUTPUT.PUT_LINE('CGPA is always increasing');
57 ELSIF v_is_dec THEN
58   DBMS_OUTPUT.PUT_LINE('CGPA is always decreasing');
59 ELSE
60   DBMS_OUTPUT.PUT_LINE('CGPA is following a zig-zag pattern');
61 END IF;
62 END CHECK_CGPA_TREND;
63 /

```

4.3 20 – A2 + B2 (Trigger) Online

20-A2 + B2 (Trigger): Validation & Salary History Triggers

1. Trigger on EMPLOYEES INSERT:

- Email format: LAST_NAME@orcl.com (lowercase). If not, change to LAST_NAME.JOB_ID@orcl.com.
- Phone format: 5555-EMPLOYEE_ID-DEPARTMENT_ID (use 000 if dept null).
- Salary must be positive; if negative, raise application error -20100.

2. Salary History trigger on INSERT, UPDATE, or DELETE logging to Salary_History.

```

1  -- 1. Employee Validation Trigger
2  CREATE OR REPLACE TRIGGER trg_employee_insert_val
3  BEFORE INSERT ON employees
4  FOR EACH ROW
5  DECLARE
6      v_expected_email VARCHAR2(100);
7      v_dept_str VARCHAR2(10);
8  BEGIN
9      -- Salary validation
10     IF :NEW.salary < 0 THEN
11         RAISE_APPLICATION_ERROR(-20100, 'Negative Value in Salary Field. Insertion
12         Aborted');
13     END IF;
14
15     -- Email formatting
16     v_expected_email := LOWER(:NEW.last_name || '@orcl.com');
17     IF LOWER(:NEW.email) != v_expected_email THEN
18         :NEW.email := LOWER(:NEW.last_name || '.' || :NEW.job_id || '@orcl.com');
19     END IF;
20
21     -- Phone formatting
22     IF :NEW.department_id IS NULL THEN
23         v_dept_str := '000';
24     ELSE
25         v_dept_str := TO_CHAR(:NEW.department_id);
26     END IF;
27     :NEW.phone_number := '5555-' || :NEW.employee_id || '-' || v_dept_str;
28 END;
29 /
30
31 -- 2. Salary History Table and Trigger
32 CREATE TABLE Salary_History (
33     HISTORY_ID NUMBER,
34     EMPLOYEE_ID NUMBER,
35     OLD_SALARY NUMBER,
36     NEW_SALARY NUMBER,
37     CHANGE_DATE DATE,
38     CHANGE_TYPE VARCHAR2(50)
39 );
40
41 CREATE SEQUENCE Salary_History_Seq START WITH 1 INCREMENT BY 1;
42
43 CREATE OR REPLACE TRIGGER trg_salary_history
44 AFTER INSERT OR UPDATE OR DELETE ON employees
45 FOR EACH ROW
46 BEGIN
47     IF INSERTING THEN
48         INSERT INTO Salary_History (HISTORY_ID, EMPLOYEE_ID, OLD_SALARY, NEW_SALARY,
49         CHANGE_DATE, CHANGE_TYPE)
50         VALUES (Salary_History_Seq.NEXTVAL, :NEW.employee_id, NULL, :NEW.salary,
51         SYSDATE, 'INSERT');
52     ELSIF UPDATING THEN
53         IF :OLD.salary != :NEW.salary OR (:OLD.salary IS NULL AND :NEW.salary IS NOT
54         NULL) THEN
55             INSERT INTO Salary_History (HISTORY_ID, EMPLOYEE_ID, OLD_SALARY,

```

```
51         NEW_SALARY, CHANGE_DATE, CHANGE_TYPE)
52         VALUES (Salary_History_Seq.NEXTVAL, :NEW.employee_id, :OLD.salary,
53                 :NEW.salary, SYSDATE, 'UPDATE');
54     END IF;
55     ELSIF DELETING THEN
56         INSERT INTO Salary_History (HISTORY_ID, EMPLOYEE_ID, OLD_SALARY, NEW_SALARY,
57                                     CHANGE_DATE, CHANGE_TYPE)
58         VALUES (Salary_History_Seq.NEXTVAL, :OLD.employee_id, :OLD.salary, NULL,
59                 SYSDATE, 'DELETE');
60     END IF;
61 END;
62 /
```

4.4 20 – A2 + B2 Online

20-A2 + B2: Department Count Function & CGPA Procedure

1. Function HAS_ENROLLED_GT_10_DEPTS(p_sid): returns TRUE if student attended courses of > 10 departments, else FALSE.

Anonymous block: loop over students with exactly 125 credits in descending order of ID. Cannot declare any variables.

2. Procedure GET_STUDENT_CGPA(p_sid, p_cgpa OUT): computes CGPA using maximum grade point per course.

```

1  -- 1a: Function
2  CREATE OR REPLACE FUNCTION HAS_ENROLLED_GT_10_DEPTS(p_sid IN VARCHAR2) RETURN BOOLEAN
3  IS
4      v_dept_count NUMBER;
5  BEGIN
6      SELECT COUNT(DISTINCT c.dept_name) INTO v_dept_count
7      FROM takes t
8      JOIN course c ON t.course_id = c.course_id
9      WHERE t.id = p_sid;
10     RETURN (v_dept_count > 10);
11 END HAS_ENROLLED_GT_10_DEPTS;
12 /
13 -- 1b: Anonymous Block (Zero Variables Declared)
14 BEGIN
15     FOR r IN (
16         SELECT id, tot_cred
17         FROM student
18         WHERE tot_cred = 125
19         ORDER BY TO_NUMBER(id) DESC
20     ) LOOP
21         IF HAS_ENROLLED_GT_10_DEPTS(r.id) THEN
22             DBMS_OUTPUT.PUT_LINE('Student ID: ' || r.id || ' has enrolled in more than
23                 ten departments.');

```

```
51         END) AS best_gp
52     FROM takes t
53     JOIN course c ON t.course_id = c.course_id
54     WHERE t.id = p_sid AND t.grade IS NOT NULL
55     GROUP BY t.course_id, c.credits
56 );
57 EXCEPTION
58     WHEN ZERO_DIVIDE THEN
59         p_cgpa := 0;
60     WHEN NO_DATA_FOUND THEN
61         p_cgpa := 0;
62 END GET_STUDENT_CGPA;
63 /
```

4.5 20 – B1 (Trigger) Online

20-B1 (Trigger): Job Salary Range Bounds & Auto Manager Assignment

1. Trigger on EMPLOYEES: When SALARY or JOB_ID is updated, check if new salary is within MIN_SALARY and MAX_SALARY of the new job. If below, set to MIN_SALARY. If above, set to MAX_SALARY. If JOB_ID is changed, ensure MIN_SALARY of new job is $\geq$ MIN_SALARY of old job; otherwise abort update.
2. Trigger on EMPLOYEES INSERT: When MANAGER_ID is null, print message and assign the department's manager.

```

1  -- Question 1
2  CREATE OR REPLACE TRIGGER trg_salary_job_bounds
3  BEFORE UPDATE OF salary, job_id ON employees
4  FOR EACH ROW
5  DECLARE
6      v_old_min jobs.min_salary%TYPE;
7      v_new_min jobs.min_salary%TYPE;
8      v_new_max jobs.max_salary%TYPE;
9  BEGIN
10     SELECT min_salary, max_salary INTO v_new_min, v_new_max
11     FROM jobs WHERE job_id = :NEW.job_id;
12
13     IF :OLD.job_id != :NEW.job_id THEN
14         SELECT min_salary INTO v_old_min FROM jobs WHERE job_id = :OLD.job_id;
15         IF v_new_min < v_old_min THEN
16             RAISE_APPLICATION_ERROR(-20003, 'Cannot change to a job with lower minimum
17             salary. ');
18         END IF;
19     END IF;
20
21     IF :NEW.salary < v_new_min THEN
22         :NEW.salary := v_new_min;
23     ELSIF :NEW.salary > v_new_max THEN
24         :NEW.salary := v_new_max;
25     END IF;
26 END;
27 /
28
29 -- Question 2
30 CREATE OR REPLACE TRIGGER trg_auto_assign_dept_mgr
31 BEFORE INSERT ON employees
32 FOR EACH ROW
33 DECLARE
34     v_dept_mgr departments.manager_id%TYPE;
35 BEGIN
36     IF :NEW.manager_id IS NULL AND :NEW.department_id IS NOT NULL THEN
37         DBMS_OUTPUT.PUT_LINE('No manager of "' || :NEW.first_name || '" has been
38         assigned');
39         SELECT manager_id INTO v_dept_mgr
40         FROM departments
41         WHERE department_id = :NEW.department_id;
42         :NEW.manager_id := v_dept_mgr;
43     END IF;
44 EXCEPTION
45     WHEN NO_DATA_FOUND THEN
46         NULL; -- department has no manager
47 END;
48 /

```

4.6 20 – B1 Online

20-B1: Semester Course Count & Instructor Increment

- 1a. Function COMPARE_SEMESTERS(p_sid): returns +13 if Fall courses > Spring courses, -7 if Fall < Spring, 0 otherwise.
- 1b. Anonymous block: Loop over advisees of instructor 90643 in ascending order of student ID. No INTO keyword. Call function exactly once per loop.
2. Procedure: Increase instructor salary by SC% where SC = avg credits taught per year. No increment if previous salary > \$125,000. Max 2 SELECTs, no joining 3+ tables, declare only 1 variable.

```

1  -- 1a: Function
2  CREATE OR REPLACE FUNCTION COMPARE_SEMESTERS(p_sid IN VARCHAR2) RETURN NUMBER IS
3      v_fall NUMBER := 0;
4      v_spring NUMBER := 0;
5  BEGIN
6      SELECT COUNT(*) INTO v_fall FROM takes WHERE id = p_sid AND semester = 'Fall';
7      SELECT COUNT(*) INTO v_spring FROM takes WHERE id = p_sid AND semester = 'Spring';
8
9      IF v_fall > v_spring THEN
10         RETURN 13;
11     ELSIF v_fall < v_spring THEN
12         RETURN -7;
13     ELSE
14         RETURN 0;
15     END IF;
16 END COMPARE_SEMESTERS;
17 /
18
19 -- 1b: Anonymous Block (No INTO keyword)
20 BEGIN
21     FOR r IN (
22         SELECT s_id
23         FROM advisor
24         WHERE i_id = '90643'
25         ORDER BY TO_NUMBER(s_id) ASC
26     ) LOOP
27         CASE COMPARE_SEMESTERS(r.s_id)
28             WHEN 13 THEN
29                 DBMS_OUTPUT.PUT_LINE('Student ID: ' || r.s_id || ' has more courses in
30                     fall semesters.');

```

```
50         FROM teaches t
51         JOIN course c ON t.course_id = c.course_id
52         GROUP BY t.id
53     ) sc_data ON i.id = sc_data.id
54     ORDER BY i.name ASC
55 ) LOOP
56     IF rec.salary <= 125000 THEN
57         v_new_sal := ROUND(rec.salary * (1 + rec.sc / 100), 2);
58         UPDATE instructor SET salary = v_new_sal WHERE id = rec.id;
59     ELSE
60         v_new_sal := rec.salary;
61     END IF;
62
63     DBMS_OUTPUT.PUT_LINE('Salary of ' || rec.name || ' has increased from ' ||
64                           rec.salary || ' to ' || v_new_sal);
65 END LOOP;
66 COMMIT;
67 END INCREASE_INSTRUCTOR_SALARY;
68 /
```

5 2021 Batch Questions & Solutions

5.1 21 – A Online

21-A Question 1: RANK_JOBS Procedure

Table JOB_RANK(JOB_ID VARCHAR2(10), RANK NUMBER). Takes MIN_HIRED_COUNT. Ranks jobs where JOB_ID matches extracted ID from JOB_TITLE (first 2 letters of first word + '_' + first 3 letters of second word). E.g. 'Sales Representative' → 'SA_REP'. Matching jobs with hired count $\geq$ MIN_HIRED_COUNT are ranked by avg salary descending. Insert into JOB_RANK.

```

1 CREATE TABLE JOB_RANK (
2     JOB_ID VARCHAR2(10),
3     RANK NUMBER
4 );
5
6 CREATE OR REPLACE PROCEDURE RANK_JOBS(MIN_HIRED_COUNT IN NUMBER) IS
7     v_rank NUMBER := 0;
8     CURSOR c_jobs IS
9         SELECT j.job_id, j.job_title, COUNT(e.employee_id) AS hired_cnt, AVG(e.salary)
10            AS avg_sal
11        FROM jobs j
12       JOIN employees e ON j.job_id = e.job_id
13      WHERE j.job_id = UPPER(
14            SUBSTR(REGEXP_SUBSTR(j.job_title, '^([A-Za-z]+)'), 1, 2) || '_' ||
15            SUBSTR(REGEXP_SUBSTR(j.job_title, '\s+([A-Za-z]+)'), 1, 1, NULL, 1), 1, 3)
16        )
17      GROUP BY j.job_id, j.job_title
18     HAVING COUNT(e.employee_id) >= MIN_HIRED_COUNT
19     ORDER BY AVG(e.salary) DESC;
20 BEGIN
21     EXECUTE IMMEDIATE 'TRUNCATE TABLE JOB_RANK';
22
23     FOR r IN c_jobs LOOP
24         v_rank := v_rank + 1;
25         INSERT INTO JOB_RANK(JOB_ID, RANK) VALUES (r.job_id, v_rank);
26         DBMS_OUTPUT.PUT_LINE('RANK-' || v_rank);
27         DBMS_OUTPUT.PUT_LINE(r.job_title || '(' || r.job_id || ')');
28         DBMS_OUTPUT.PUT_LINE('EMPLOYEES HIRED: ' || r.hired_cnt);
29         DBMS_OUTPUT.PUT_LINE('AVG SALARY: ' || ROUND(r.avg_sal, 2));
30     END LOOP;
31     COMMIT;
32 END RANK_JOBS;

```

21-A Question 2: Validate Salary Updates in TEMP_EMPLOYEES

Trigger on TEMP_EMPLOYEES when SALARY is updated. Validate using original EMPLOYEES table: - Case 1: Salary cannot be less than minimum salary of same JOB_ID of other employees. - Case 2: If employee is not a manager, salary must be less than minimum salary of managers who manage this JOB_ID. - Case 3: If employee is a manager, salary must be less than salary of the manager of the same department.

```

1 CREATE TABLE TEMP_EMPLOYEES AS SELECT * FROM employees;
2
3 CREATE OR REPLACE TRIGGER trg_validate_temp_emp_sal
4 BEFORE UPDATE OF salary ON TEMP_EMPLOYEES
5 FOR EACH ROW
6 DECLARE
7     v_min_job_sal NUMBER;
8     v_min_mgr_sal NUMBER;

```

```

9      v_dept_mgr_sal NUMBER;
10     v_is_mgr NUMBER;
11 BEGIN
12     -- Case 1: Min salary of other employees in same job
13     SELECT NVL(MIN(salary), :NEW.salary) INTO v_min_job_sal
14     FROM employees
15     WHERE job_id = :NEW.job_id AND employee_id != :NEW.employee_id;
16
17     IF :NEW.salary < v_min_job_sal THEN
18         RAISE_APPLICATION_ERROR(-20004, 'Salary cannot be less than minimum salary of
19         peers in the same job.');
```

20

```

19     END IF;
20
21     -- Check if this employee is a manager
22     SELECT COUNT(*) INTO v_is_mgr FROM employees WHERE manager_id = :NEW.employee_id;
23
24     IF v_is_mgr = 0 THEN
25         -- Case 2: Non-manager must earn less than min salary of managers managing
26         -- this job
27         SELECT MIN(m.salary) INTO v_min_mgr_sal
28         FROM employees m
29         JOIN employees e ON e.manager_id = m.employee_id
30         WHERE e.job_id = :NEW.job_id;
31
32         IF v_min_mgr_sal IS NOT NULL AND :NEW.salary >= v_min_mgr_sal THEN
33             RAISE_APPLICATION_ERROR(-20005, 'Non-manager salary must be less than
34             minimum manager salary for this job.');
```

35

```

33         END IF;
34     ELSE
35         -- Case 3: Manager must earn less than the department manager
36         SELECT m.salary INTO v_dept_mgr_sal
37         FROM departments d
38         JOIN employees m ON d.manager_id = m.employee_id
39         WHERE d.department_id = :NEW.department_id;
40
41         IF v_dept_mgr_sal IS NOT NULL AND :NEW.employee_id != d.manager_id AND
42         :NEW.salary >= v_dept_mgr_sal THEN
43             RAISE_APPLICATION_ERROR(-20006, 'Manager salary must be less than
44             department head salary.');
```

45

```

43         END IF;
44     END IF;
45 EXCEPTION
46     WHEN NO_DATA_FOUND THEN NULL;
47 END;
48 /
```

5.2 21 – B Online

21-B: Lowest Paid Employees & LPET Tag Trigger

Table: lowest_paid_employees(employee_id, department_id, saldiff, bonus, tag).

Task 1: Procedure Update_Lowest_Paid_Employees(m, n): Populates lowest salaried permanent employee (> 1 yr service) per department. If tie, earliest join date. If lowest = highest in department, skip. $saldiff = \max_dept_salary - salary$. Bonus: m per n full years worked +5000 if manager of ≥ 5 subordinates. Tag inserted null.

Task 2: Trigger Update_LPET_Tag_Trigger: Row-level BEFORE INSERT on lowest_paid_employees. Check $saldiff \leq \max$ global saldiff among departments; else throw error. Sets tag: $< 10000 \rightarrow$ 'low', $[10000, 20000) \rightarrow$ 'very low', $\geq 20000 \rightarrow$ 'extremely low'.

```

1 CREATE TABLE lowest_paid_employees (
2     employee_id VARCHAR2(100),
3     department_id VARCHAR2(100),
4     saldiff NUMBER,
5     bonus NUMBER,
6     tag VARCHAR2(100)
7 );
8
9 -- Task 2: Trigger
10 CREATE OR REPLACE TRIGGER Update_LPET_Tag_Trigger
11 BEFORE INSERT ON lowest_paid_employees
12 FOR EACH ROW
13 DECLARE
14     v_max_global_saldiff NUMBER;
15 BEGIN
16     SELECT MAX(max_sal - min_sal) INTO v_max_global_saldiff
17     FROM (
18         SELECT MAX(salary) AS max_sal, MIN(salary) AS min_sal
19         FROM employees
20         WHERE department_id IS NOT NULL
21         GROUP BY department_id
22     );
23
24     IF :NEW.saldiff > v_max_global_saldiff THEN
25         RAISE_APPLICATION_ERROR(-20007, 'Sanity check failed: saldiff exceeds max
26             global departmental saldiff.');
```

26 END IF;

```

27
28     IF :NEW.saldiff < 10000 THEN
29         :NEW.tag := 'low';
30     ELSIF :NEW.saldiff < 20000 THEN
31         :NEW.tag := 'very low';
32     ELSE
33         :NEW.tag := 'extremely low';
34     END IF;
35 END;
36 /
37
38 -- Task 1: Procedure
39 CREATE OR REPLACE PROCEDURE Update_Lowest_Paid_Employees(m IN NUMBER, n IN NUMBER) IS
40     v_max_dept_sal NUMBER;
41     v_min_dept_sal NUMBER;
42     v_bonus NUMBER;
43     v_years NUMBER;
44     v_sub_count NUMBER;
45 BEGIN
46     EXECUTE IMMEDIATE 'TRUNCATE TABLE lowest_paid_employees';
47
48     FOR dept IN (SELECT DISTINCT department_id FROM employees WHERE department_id IS
49         NOT NULL) LOOP
```

```
49      SELECT MAX(salary), MIN(salary)
50      INTO v_max_dept_sal, v_min_dept_sal
51      FROM employees
52      WHERE department_id = dept.department_id
53             AND MONTHS_BETWEEN(SYSDATE, hire_date) / 12 > 1;
54
55      -- Check Case 3: lowest cannot equal highest
56      IF v_max_dept_sal IS NOT NULL AND v_min_dept_sal IS NOT NULL AND
          v_max_dept_sal != v_min_dept_sal THEN
57          FOR emp IN (
58              SELECT employee_id, salary, hire_date
59              FROM employees
60              WHERE department_id = dept.department_id
61                     AND salary = v_min_dept_sal
62                     AND MONTHS_BETWEEN(SYSDATE, hire_date) / 12 > 1
63              ORDER BY hire_date ASC
64          ) LOOP
65              v_years := FLOOR(MONTHS_BETWEEN(SYSDATE, emp.hire_date) / 12);
66              v_bonus := FLOOR(v_years / n) * m;
67
68              SELECT COUNT(*) INTO v_sub_count FROM employees WHERE manager_id =
                  emp.employee_id;
69              IF v_sub_count >= 5 THEN
70                  v_bonus := v_bonus + 5000;
71              END IF;
72
73              INSERT INTO lowest_paid_employees(employee_id, department_id, saldiff,
                  bonus, tag)
74              VALUES (emp.employee_id, dept.department_id, v_max_dept_sal -
                  emp.salary, v_bonus, NULL);
75
76              EXIT; -- Only 1 lowest-paid employee inserted per dept
77          END LOOP;
78      END IF;
79  END LOOP;
80  COMMIT;
81  END Update_Lowest_Paid_Employees;
82  /
```

5.3 21 – C Online

21-C: Manager Credentials & Job Transfer Department Trigger

Problem 1: Procedure **POPULATE_CREDS**(MIN_EMP_COUNT, MIN_JOB_COUNT): Populates MANAGER_CRED in TEMP_EMPLOYEES as Substr(email,1,2)**Substr(email,-2)-SubordinatesCount. Eligible if $\geq$ MIN_JOB_COUNT distinct jobs held, OR currently manages $\geq$ MIN_EMP_COUNT subordinates.

Problem 2: Trigger on TEMP_EMPLOYEES: When JOB_ID is updated: Rank calculated by max salary of employees in that job. If new rank > current rank $\rightarrow$ valid. If new rank < current rank $\rightarrow$ valid only if employee served < 5 years. Update DEPARTMENT_ID from DEPARTMENT_JOB_MAP_TABLE.

```

1 CREATE TABLE TEMP_EMPLOYEES (
2     EMPLOYEE_ID VARCHAR2(10),
3     NAME VARCHAR2(100),
4     EMAIL VARCHAR2(50),
5     MANAGER_ID NUMBER(4,0),
6     MANAGER_CRED VARCHAR2(20)
7 );
8
9 -- Problem 1: Procedure
10 CREATE OR REPLACE PROCEDURE POPULATE_CREDS(MIN_EMP_COUNT IN NUMBER, MIN_JOB_COUNT IN
11     NUMBER) IS
12     v_job_count NUMBER;
13     v_emp_count NUMBER;
14     v_cred VARCHAR2(30);
15 BEGIN
16     FOR mgr IN (
17         SELECT DISTINCT e.manager_id AS mid, m.email
18         FROM employees e
19         JOIN employees m ON e.manager_id = m.employee_id
20         WHERE e.manager_id IS NOT NULL
21     ) LOOP
22         -- Distinct jobs served (current + history)
23         SELECT COUNT(DISTINCT jid) INTO v_job_count
24         FROM (
25             SELECT job_id AS jid FROM employees WHERE employee_id = mgr.mid
26             UNION
27             SELECT job_id AS jid FROM job_history WHERE employee_id = mgr.mid
28         );
29         SELECT COUNT(*) INTO v_emp_count FROM employees WHERE manager_id = mgr.mid;
30
31         IF v_job_count >= MIN_JOB_COUNT OR v_emp_count >= MIN_EMP_COUNT THEN
32             v_cred := SUBSTR(mgr.email, 1, 2) || '**' || SUBSTR(mgr.email, -2) || '-' ||
33                 || v_emp_count;
34             UPDATE TEMP_EMPLOYEES
35             SET MANAGER_CRED = v_cred
36             WHERE EMPLOYEE_ID = TO_CHAR(mgr.mid);
37         END IF;
38     END LOOP;
39     COMMIT;
40 END POPULATE_CREDS;
41 /
42
43 -- Problem 2: Job Transfer Trigger
44 CREATE OR REPLACE TRIGGER trg_job_transfer_validate
45 BEFORE UPDATE OF JOB_ID ON TEMP_EMPLOYEES
46 FOR EACH ROW
47 DECLARE
48     v_old_max_sal NUMBER;

```

```
48     v_new_max_sal NUMBER;
49     v_years NUMBER;
50     v_new_dept NUMBER;
51 BEGIN
52     SELECT MAX(salary) INTO v_old_max_sal FROM employees WHERE job_id = :OLD.JOB_ID;
53     SELECT MAX(salary) INTO v_new_max_sal FROM employees WHERE job_id = :NEW.JOB_ID;
54
55     SELECT FLOOR(MONTHS_BETWEEN(SYSDATE, hire_date) / 12) INTO v_years
56     FROM employees WHERE employee_id = TO_NUMBER(:OLD.EMPLOYEE_ID);
57
58     IF v_new_max_sal > v_old_max_sal THEN
59         NULL; -- Valid
60     ELSEIF v_years < 5 THEN
61         NULL; -- Valid lower rank because service < 5 years
62     ELSE
63         RAISE_APPLICATION_ERROR(-20008, 'Transfer not allowed: Lower rank job and
64         service >= 5 years.');
```

```
65
66     -- Lookup department
67     SELECT department_id INTO v_new_dept
68     FROM DEPARTMENT_JOB_MAP_TABLE
69     WHERE job_id = :NEW.JOB_ID AND ROWNUM = 1;
70
71     :NEW.DEPARTMENT_ID := v_new_dept;
72 EXCEPTION
73     WHEN NO_DATA_FOUND THEN NULL;
74 END;
75 /
```

5.4 21 – Practice

21-Practice: Top N Paid, Longest Serving In Job, Transfer Procedure

Task 1: Function check whether employee is among top N paid in their department (returns 1/0).

Task 2: Function find longest-serving employee's last name in a given job_title.

Task 3: Procedure transfer(eid, job_id): Changes job_id. New salary = lowest salary among employees currently in that job who joined earlier than eid and earn more than current salary of eid. If none, salary remains unchanged. Prints new salary.

```

1  -- Task 1
2  CREATE OR REPLACE FUNCTION IS_TOP_N_PAID(p_eid IN NUMBER, p_n IN NUMBER) RETURN NUMBER
3  IS
4      v_rank NUMBER;
5  BEGIN
6      SELECT rnk INTO v_rank
7      FROM (
8          SELECT employee_id, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
9          FROM employees
10         WHERE department_id = (SELECT department_id FROM employees WHERE employee_id =
11                                 p_eid)
12     )
13     WHERE employee_id = p_eid;
14
15     IF v_rank <= p_n THEN RETURN 1; ELSE RETURN 0; END IF;
16 EXCEPTION WHEN OTHERS THEN RETURN 0;
17 END IS_TOP_N_PAID;
18 /
19
20 -- Task 2
21 CREATE OR REPLACE FUNCTION LONGEST_SERVING_IN_JOB(p_job_title IN VARCHAR2) RETURN
22 VARCHAR2 IS
23     v_lname employees.last_name%TYPE;
24 BEGIN
25     SELECT e.last_name INTO v_lname
26     FROM employees e
27     JOIN jobs j ON e.job_id = j.job_id
28     WHERE UPPER(j.job_title) = UPPER(p_job_title)
29     ORDER BY e.hire_date ASC
30     FETCH FIRST 1 ROW ONLY;
31
32     RETURN v_lname;
33 EXCEPTION WHEN OTHERS THEN RETURN 'NOT FOUND';
34 END LONGEST_SERVING_IN_JOB;
35 /
36
37 -- Task 3
38 CREATE OR REPLACE PROCEDURE transfer(p_eid IN NUMBER, p_new_job_id IN VARCHAR2) IS
39     v_cur_sal employees.salary%TYPE;
40     v_cur_hire DATE;
41     v_new_sal employees.salary%TYPE;
42 BEGIN
43     SELECT salary, hire_date INTO v_cur_sal, v_cur_hire FROM employees WHERE
44         employee_id = p_eid;
45
46     SELECT MIN(salary) INTO v_new_sal
47     FROM employees
48     WHERE job_id = p_new_job_id
49           AND hire_date < v_cur_hire
50           AND salary > v_cur_sal;
51
52     IF v_new_sal IS NULL THEN
53         v_new_sal := v_cur_sal;
54     END IF;

```

```
51
52     UPDATE employees
53     SET job_id = p_new_job_id, salary = v_new_sal
54     WHERE employee_id = p_eid;
55
56     DBMS_OUTPUT.PUT_LINE('Employee ' || p_eid || ' transferred to ' || p_new_job_id ||
57                           ' with salary ' || v_new_sal);
57 END transfer;
58 /
```

6 2022 Batch Questions & Solutions

6.1 22 – A Online

22-A Questions 1, 2, 3: Longest Serving, Rank Jobs, Leaves Trigger

Q1: Procedure LONGEST_SERVING_EMPLOYEE(REGION_NAME): Displays Name, Job Title, Hire Date, Country, Department, City of longest-serving employee in the region.

Q2: Procedure RANK_JOBS: Ranks jobs descending by employee count; ties broken by descending average salary. Prints details.

Q3: Trigger on Employee DELETE: Populates table Leaves(employee_id, employee_working_instead, date). Substitute rule: If employee is manager → manager with closest subordinate count; else if has manager → peer under same manager with closest salary.

```

1  -- Q1: LONGEST_SERVING_EMPLOYEE
2  CREATE OR REPLACE PROCEDURE LONGEST_SERVING_EMPLOYEE(p_region_name IN VARCHAR2) IS
3      r_name VARCHAR2(100);
4      r_job VARCHAR2(100);
5      r_hire DATE;
6      r_country VARCHAR2(100);
7      r_dept VARCHAR2(100);
8      r_city VARCHAR2(100);
9  BEGIN
10     SELECT e.first_name || ' ' || e.last_name, j.job_title, e.hire_date,
11            c.country_name, d.department_name, l.city
12     INTO r_name, r_job, r_hire, r_country, r_dept, r_city
13     FROM employees e
14     JOIN jobs j ON e.job_id = j.job_id
15     JOIN departments d ON e.department_id = d.department_id
16     JOIN locations l ON d.location_id = l.location_id
17     JOIN countries c ON l.country_id = c.country_id
18     JOIN regions r ON c.region_id = r.region_id
19     WHERE UPPER(r.region_name) = UPPER(p_region_name)
20     ORDER BY e.hire_date ASC
21     FETCH FIRST 1 ROW ONLY;
22
23     DBMS_OUTPUT.PUT_LINE('Longest Serving Employee in Region: ' || p_region_name);
24     DBMS_OUTPUT.PUT_LINE('Name: ' || r_name);
25     DBMS_OUTPUT.PUT_LINE('Job Title: ' || r_job);
26     DBMS_OUTPUT.PUT_LINE('Hire Date: ' || TO_CHAR(r_hire, 'DD-MON-YYYY'));
27     DBMS_OUTPUT.PUT_LINE('Country: ' || r_country);
28     DBMS_OUTPUT.PUT_LINE('Department: ' || r_dept);
29     DBMS_OUTPUT.PUT_LINE('City: ' || r_city);
30 EXCEPTION
31     WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No employee found in region ' ||
32     p_region_name);
33 END LONGEST_SERVING_EMPLOYEE;
34 /
35
36 -- Q2: RANK_JOBS
37 CREATE OR REPLACE PROCEDURE RANK_JOBS IS
38     v_rank NUMBER := 0;
39 BEGIN
40     FOR r IN (
41         SELECT j.job_title, COUNT(e.employee_id) AS emp_count,
42                ROUND(AVG(e.salary), 2) AS avg_sal,
43                MAX(e.salary) AS max_sal, MIN(e.salary) AS min_sal
44         FROM jobs j
45         JOIN employees e ON j.job_id = e.job_id
46         GROUP BY j.job_title
47         ORDER BY COUNT(e.employee_id) DESC, AVG(e.salary) DESC
48     ) LOOP

```

```

47         v_rank := v_rank + 1;
48         DBMS_OUTPUT.PUT_LINE('Rank ' || v_rank);
49         DBMS_OUTPUT.PUT_LINE('Job Title: ' || r.job_title);
50         DBMS_OUTPUT.PUT_LINE('Employees: ' || r.emp_count);
51         DBMS_OUTPUT.PUT_LINE('Avg Salary: ' || r.avg_sal);
52         DBMS_OUTPUT.PUT_LINE('Max Salary: ' || r.max_sal);
53         DBMS_OUTPUT.PUT_LINE('Min Salary: ' || r.min_sal);
54     END LOOP;
55 END RANK_JOBS;
56 /
57
58 -- Q3: Leaves Trigger
59 CREATE TABLE Leaves (
60     employee_id NUMBER,
61     employee_working_instead NUMBER,
62     leave_date DATE
63 );
64
65 CREATE OR REPLACE TRIGGER trg_leaves
66 AFTER DELETE ON employees
67 FOR EACH ROW
68 DECLARE
69     PRAGMA AUTONOMOUS_TRANSACTION;
70     v_is_mgr NUMBER;
71     v_sub NUMBER;
72     v_sub_count NUMBER;
73 BEGIN
74     -- Check if deleted employee was a manager
75     SELECT COUNT(*) INTO v_sub_count FROM employees WHERE manager_id =
76         :OLD.employee_id;
77
78     IF v_sub_count > 0 THEN
79         -- Find manager with closest subordinate count
80         SELECT mgr_id INTO v_sub
81         FROM (
82             SELECT manager_id AS mgr_id, ABS(COUNT(*) - v_sub_count) AS diff
83             FROM employees
84             WHERE manager_id IS NOT NULL AND manager_id != :OLD.employee_id
85             GROUP BY manager_id
86             ORDER BY diff ASC
87         ) WHERE ROWNUM = 1;
88     ELSIF :OLD.manager_id IS NOT NULL THEN
89         -- Find peer under same manager with closest salary
90         SELECT employee_id INTO v_sub
91         FROM (
92             SELECT employee_id, ABS(salary - :OLD.salary) AS diff
93             FROM employees
94             WHERE manager_id = :OLD.manager_id AND employee_id != :OLD.employee_id
95             ORDER BY diff ASC
96         ) WHERE ROWNUM = 1;
97     ELSE
98         v_sub := NULL;
99     END IF;
100
101     INSERT INTO Leaves VALUES (:OLD.employee_id, v_sub, SYSDATE);
102     COMMIT;
103 EXCEPTION
104     WHEN NO_DATA_FOUND THEN
105         INSERT INTO Leaves VALUES (:OLD.employee_id, NULL, SYSDATE);
106         COMMIT;
107 END;
108 /

```

6.2 22 – B Online

22-B Questions 1, 2, 3: Exchange Employees, Location Salary Report, Transfers Trigger

Q1: Function Exchange_Employees(m1, m2): In EMPLOYEES_COPY, exchange lowest-paid employees under two managers. They switch departments and managers; salaries increase by 50% of original salary difference.

Q2: Procedure LOCATION_SALARY_REPORT: Group by city, calculate employee count, avg salary, and highest-paid job title. Rank by employee count ASC, then avg salary DESC.

Q3: Trigger on UPDATE OF department_id: Table Transfers(employee_id, employee_working_instead, new_department, transfer_date). Work-in-place gets +0.5× transferred employee's salary. Transferred employee's manager updated to manager in new department with closest subordinate count.

```

1  -- Q1: Exchange_Employees
2  CREATE TABLE employees_copy AS SELECT * FROM employees;
3
4  CREATE OR REPLACE FUNCTION Exchange_Employees(m1 IN NUMBER, m2 IN NUMBER) RETURN
   VARCHA2 IS
5      e1 employees_copy%ROWTYPE;
6      e2 employees_copy%ROWTYPE;
7      v_diff NUMBER;
8  BEGIN
9      SELECT * INTO e1 FROM (
10         SELECT * FROM employees_copy WHERE manager_id = m1 ORDER BY salary ASC,
            hire_date ASC
11     ) WHERE ROWNUM = 1;
12
13     SELECT * INTO e2 FROM (
14         SELECT * FROM employees_copy WHERE manager_id = m2 ORDER BY salary ASC,
            hire_date ASC
15     ) WHERE ROWNUM = 1;
16
17     v_diff := ABS(e1.salary - e2.salary) * 0.5;
18
19     DBMS_OUTPUT.PUT_LINE('Before EXCHANGE EMPLOYEES (' || m1 || ', ' || m2 || '):');
20     DBMS_OUTPUT.PUT_LINE('Employee ID: ' || e1.employee_id || ' Name: ' ||
        e1.first_name || ' ' || e1.last_name || ' Sal: ' || e1.salary);
21     DBMS_OUTPUT.PUT_LINE('Employee ID: ' || e2.employee_id || ' Name: ' ||
        e2.first_name || ' ' || e2.last_name || ' Sal: ' || e2.salary);
22
23     -- Perform exchange
24     UPDATE employees_copy
25     SET manager_id = m2, department_id = e2.department_id, salary = salary + v_diff
26     WHERE employee_id = e1.employee_id;
27
28     UPDATE employees_copy
29     SET manager_id = m1, department_id = e1.department_id, salary = salary + v_diff
30     WHERE employee_id = e2.employee_id;
31
32     DBMS_OUTPUT.PUT_LINE('After EXCHANGE EMPLOYEES (' || m1 || ', ' || m2 || '):');
33     DBMS_OUTPUT.PUT_LINE('Employee ID: ' || e1.employee_id || ' New Mgr: ' || m2 || '
        New Sal: ' || (e1.salary + v_diff));
34     DBMS_OUTPUT.PUT_LINE('Employee ID: ' || e2.employee_id || ' New Mgr: ' || m1 || '
        New Sal: ' || (e2.salary + v_diff));
35     RETURN 'SUCCESS';
36 EXCEPTION
37     WHEN NO_DATA_FOUND THEN
38         DBMS_OUTPUT.PUT_LINE('One or both managers have no subordinates. ');
39         RETURN 'FAILED';
40 END Exchange_Employees;
41 /
42

```

```

43 -- Q2: LOCATION_SALARY_REPORT
44 CREATE OR REPLACE PROCEDURE LOCATION_SALARY_REPORT IS
45     v_rank NUMBER := 0;
46 BEGIN
47     FOR r IN (
48         SELECT l.city, COUNT(e.employee_id) AS emp_count,
49                ROUND(AVG(e.salary), 2) AS avg_sal,
50                (SELECT j.job_title FROM employees e2 JOIN jobs j ON e2.job_id =
51                 j.job_id
52                 JOIN departments d2 ON e2.department_id = d2.department_id
53                 WHERE d2.location_id = l.location_id
54                 ORDER BY e2.salary DESC FETCH FIRST 1 ROW ONLY) AS top_job
55         FROM locations l
56         JOIN departments d ON l.location_id = d.location_id
57         JOIN employees e ON d.department_id = e.department_id
58         GROUP BY l.city, l.location_id
59         ORDER BY COUNT(e.employee_id) ASC, AVG(e.salary) DESC
60     ) LOOP
61         v_rank := v_rank + 1;
62         DBMS_OUTPUT.PUT_LINE('Rank: ' || v_rank || ' | City: ' || r.city ||
63                               ' | Employees: ' || r.emp_count || ' | Avg Salary: ' ||
64                               r.avg_sal ||
65                               ' | Highest Paying Job: ' || r.top_job);
66     END LOOP;
67 END LOCATION_SALARY_REPORT;
68 /
69
70 -- Q3: Transfers Trigger
71 CREATE TABLE Transfers (
72     employee_id NUMBER,
73     employee_working_instead NUMBER,
74     new_department NUMBER,
75     transfer_date DATE
76 );
77
78 CREATE OR REPLACE TRIGGER trg_emp_transfer
79 AFTER UPDATE OF department_id ON employees
80 FOR EACH ROW
81 DECLARE
82     PRAGMA AUTONOMOUS_TRANSACTION;
83     v_sub_id NUMBER;
84     v_old_mgr_subs NUMBER;
85     v_new_mgr NUMBER;
86 BEGIN
87     IF :OLD.manager_id IS NOT NULL THEN
88         -- Work in place employee under same manager with closest salary
89         SELECT employee_id INTO v_sub_id
90         FROM (
91             SELECT employee_id FROM employees
92             WHERE manager_id = :OLD.manager_id AND employee_id != :OLD.employee_id
93             ORDER BY ABS(salary - :OLD.salary) ASC
94         ) WHERE ROWNUM = 1;
95
96         -- Raise substitute employee salary
97         UPDATE employees SET salary = salary + 0.5 * :OLD.salary WHERE employee_id =
98             v_sub_id;
99
100         -- Count subordinates of old manager
101         SELECT COUNT(*) INTO v_old_mgr_subs FROM employees WHERE manager_id =
102             :OLD.manager_id;
103
104         -- Find new manager in new department with closest subordinate count
105         SELECT mgr_id INTO v_new_mgr
106         FROM (
107             SELECT manager_id AS mgr_id, ABS(COUNT(*) - v_old_mgr_subs) AS diff
108             FROM employees

```

```
105         WHERE department_id = :NEW.department_id AND manager_id IS NOT NULL
106         GROUP BY manager_id
107         ORDER BY diff ASC
108     ) WHERE ROWNUM = 1;
109
110     UPDATE employees SET manager_id = v_new_mgr WHERE employee_id =
111         :NEW.employee_id;
112 END IF;
113
114 INSERT INTO Transfers VALUES (:NEW.employee_id, v_sub_id, :NEW.department_id,
115     SYSDATE);
116 COMMIT;
117 EXCEPTION
118 WHEN NO_DATA_FOUND THEN
119     INSERT INTO Transfers VALUES (:NEW.employee_id, NULL, :NEW.department_id,
120     SYSDATE);
121 COMMIT;
122 END;
123 /
```

6.3 22 – C Online

22-C Questions 1, 2, 3: Promotion Check, Salary Update, Demotion Trigger

Q1: Function IS_READY_FOR_PROMOTION(EMPLOYEE_ID): Returns YES if service ≥ 5 yrs, salary $>$ midpoint of job's min and max salary, and manages ≥ 1 subordinate. Else NO.

Q2: Procedure in EMPLOYEES_COPY: Updates salary: $Old + (Comm \times Old) + 0.1 \times MinSal + 0.1 \times DeptAvg$. No update if tenure ≤ 1 yr (relative to newest employee hire date). Cap at MaxSal.

Q3: Demotion Trigger: If salary decreased by $> 20\%$, log to Demotions. If manager, switch positions with highest-paid subordinate without switching salaries.

```

1  -- Q1: IS_READY_FOR_PROMOTION
2  CREATE OR REPLACE FUNCTION IS_READY_FOR_PROMOTION(p_eid IN NUMBER) RETURN VARCHAR2 IS
3      v_hire DATE;
4      v_sal  NUMBER;
5      v_min  NUMBER;
6      v_max  NUMBER;
7      v_subs NUMBER;
8  BEGIN
9      SELECT e.hire_date, e.salary, j.min_salary, j.max_salary
10     INTO v_hire, v_sal, v_min, v_max
11     FROM employees e
12     JOIN jobs j ON e.job_id = j.job_id
13     WHERE e.employee_id = p_eid;
14
15     SELECT COUNT(*) INTO v_subs FROM employees WHERE manager_id = p_eid;
16
17     IF MONTHS_BETWEEN(SYSDATE, v_hire) / 12 >= 5
18        AND v_sal > (v_min + v_max) / 2.0
19        AND v_subs >= 1 THEN
20         RETURN 'YES';
21     ELSE
22         RETURN 'NO';
23     END IF;
24 EXCEPTION
25     WHEN NO_DATA_FOUND THEN RETURN 'NO';
26 END IS_READY_FOR_PROMOTION;
27 /
28
29 -- Q2: Mass Salary Update Procedure
30 CREATE OR REPLACE PROCEDURE UPDATE_ALL_SALARIES IS
31     v_max_hire DATE;
32     v_dept_avg NUMBER;
33     v_new_sal  NUMBER;
34 BEGIN
35     SELECT MAX(hire_date) INTO v_max_hire FROM employees_copy;
36
37     FOR emp IN (
38         SELECT ec.employee_id, ec.salary, ec.commission_pct, ec.hire_date,
39                ec.department_id,
40                j.min_salary, j.max_salary
41         FROM employees_copy ec
42         JOIN jobs j ON ec.job_id = j.job_id
43     ) LOOP
44         IF MONTHS_BETWEEN(v_max_hire, emp.hire_date) / 12 > 1 THEN
45             SELECT NVL(AVG(salary), emp.salary) INTO v_dept_avg
46             FROM employees_copy
47             WHERE department_id = emp.department_id;
48
49             v_new_sal := emp.salary + (NVL(emp.commission_pct, 0) * emp.salary)
50                               + (0.10 * emp.min_salary) + (0.10 * v_dept_avg);

```

```

50
51         IF v_new_sal > emp.max_salary THEN
52             v_new_sal := emp.max_salary;
53         END IF;
54
55         UPDATE employees_copy SET salary = v_new_sal WHERE employee_id =
56             emp.employee_id;
57     END IF;
58 END LOOP;
59 COMMIT;
60 END UPDATE_ALL_SALARIES;
61 /
62 -- Q3: Demotion Trigger
63 CREATE TABLE Demotions (
64     employee_id NUMBER,
65     current_salary NUMBER,
66     status VARCHAR2(20),
67     demotion_date DATE
68 );
69
70 CREATE OR REPLACE TRIGGER trg_demotion
71 AFTER UPDATE OF salary ON employees
72 FOR EACH ROW
73 WHEN (NEW.salary < OLD.salary * 0.80)
74 DECLARE
75     PRAGMA AUTONOMOUS_TRANSACTION;
76     v_highest_sub employees%ROWTYPE;
77     v_newest_hire DATE;
78 BEGIN
79     SELECT MAX(hire_date) INTO v_newest_hire FROM employees;
80
81     -- Check if manager
82     BEGIN
83         SELECT * INTO v_highest_sub FROM (
84             SELECT * FROM employees WHERE manager_id = :NEW.employee_id ORDER BY
85                 salary DESC
86         ) WHERE ROWNUM = 1;
87
88         -- Demote manager: switch subordinate structure
89         UPDATE employees SET manager_id = v_highest_sub.employee_id WHERE employee_id
90             = :NEW.employee_id;
91         UPDATE employees SET manager_id = :OLD.manager_id WHERE employee_id =
92             v_highest_sub.employee_id;
93         UPDATE employees SET manager_id = v_highest_sub.employee_id
94             WHERE manager_id = :NEW.employee_id AND employee_id !=
95                 v_highest_sub.employee_id;
96     EXCEPTION
97     WHEN NO_DATA_FOUND THEN NULL; -- Not a manager
98 END;
99
100 INSERT INTO Demotions VALUES (:NEW.employee_id, :NEW.salary, 'done',
101     v_newest_hire);
102 COMMIT;
103 END;
104 /

```

7 2023 Batch Questions & Solutions

7.1 23 – A Online

23-A Question 1: Job Min/Max Salary Bound Trigger

Write a PL/SQL trigger to enforce: No employee's salary should ever exceed the maximum salary and fall below the minimum salary defined for their job in the JOBS table.

```

1 CREATE OR REPLACE TRIGGER trg_enforce_job_salary_limits
2 BEFORE INSERT OR UPDATE OF salary, job_id ON employees
3 FOR EACH ROW
4 DECLARE
5     v_min jobs.min_salary%TYPE;
6     v_max jobs.max_salary%TYPE;
7 BEGIN
8     SELECT min_salary, max_salary INTO v_min, v_max
9     FROM jobs
10    WHERE job_id = :NEW.job_id;
11
12    IF :NEW.salary < v_min OR :NEW.salary > v_max THEN
13        RAISE_APPLICATION_ERROR(-20009, 'Salary ' || :NEW.salary || ' out of allowable
14                                     range [' ||
15                                     v_min || ', ' || v_max || '] for job ' ||
16                                     :NEW.job_id);
17    END IF;
18 EXCEPTION
19     WHEN NO_DATA_FOUND THEN
20         RAISE_APPLICATION_ERROR(-20010, 'Invalid Job ID specified: ' || :NEW.job_id);
21 END;
22 /

```

23-A Question 2: Employee Transfer Procedure with Job History Logging

Takes two inputs: employee ID and new department ID.

If employee and department are valid:

- Save current job details into JOB_HISTORY table before the change.
- Move employee to new department.
- Increase salary by 15%.
- Set OUT message parameter explaining outcome.

```

1 CREATE OR REPLACE PROCEDURE TRANSFER_EMPLOYEE(
2     p_emp_id IN NUMBER,
3     p_new_dept_id IN NUMBER,
4     p_msg OUT VARCHAR2
5 ) IS
6     v_emp employees%ROWTYPE;
7     v_dept_exists NUMBER := 0;
8 BEGIN
9     SELECT * INTO v_emp FROM employees WHERE employee_id = p_emp_id;
10
11     SELECT COUNT(*) INTO v_dept_exists FROM departments WHERE department_id =
12         p_new_dept_id;
13     IF v_dept_exists = 0 THEN
14         p_msg := 'Department does not exist.';
15         RETURN;
16     END IF;
17
18     -- Archive into JOB_HISTORY
19     INSERT INTO job_history (employee_id, start_date, end_date, job_id, department_id)
20     VALUES (v_emp.employee_id, v_emp.hire_date, SYSDATE, v_emp.job_id,
21             v_emp.department_id);

```

```
20
21  -- Update Employee record
22  UPDATE employees
23  SET department_id = p_new_dept_id,
24      hire_date = SYSDATE,
25      salary = salary * 1.15
26  WHERE employee_id = p_emp_id;
27
28  COMMIT;
29  p_msg := 'Employee transferred successfully with 15% salary increase.';
30  EXCEPTION
31  WHEN NO_DATA_FOUND THEN
32      p_msg := 'Employee does not exist.';
33  WHEN OTHERS THEN
34      p_msg := 'Transfer error: ' || SQLERRM;
35  END TRANSFER_EMPLOYEE;
36  /
```

7.2 23 – B Online

23-B Question 1: Same Department for Manager and Employee Trigger

Write a PL/SQL trigger that enforces: If an employee has a manager, that manager must be in the same department as the employee.

```

1 CREATE OR REPLACE TRIGGER trg_check_mgr_dept
2 BEFORE INSERT OR UPDATE OF department_id, manager_id ON employees
3 FOR EACH ROW
4 DECLARE
5     v_mgr_dept employees.department_id%TYPE;
6 BEGIN
7     IF :NEW.manager_id IS NOT NULL AND :NEW.department_id IS NOT NULL THEN
8         SELECT department_id INTO v_mgr_dept
9         FROM employees
10        WHERE employee_id = :NEW.manager_id;
11
12        IF v_mgr_dept IS NOT NULL AND v_mgr_dept != :NEW.department_id THEN
13            RAISE_APPLICATION_ERROR(-20011, 'Manager must be in the same department as
14                employee.');
```

23-B Question 2: Salary Audit Table & Controlled Update Function

Table: SALARY_AUDIT_LOG(audit_id IDENTITY PK, employee_id, old_salary, new_salary, changed_by, change_date, reason).

Function: inputs (p_emp_id, p_pct_change, p_user, p_reason). Max increase 30%, max decrease 20%. Validate inputs, calculate new salary, update employees, insert audit log, return status message.

```

1 CREATE TABLE SALARY_AUDIT_LOG (
2     audit_id NUMBER GENERATED ALWAYS AS IDENTITY CONSTRAINT salary_audit_pk PRIMARY
3     KEY,
4     employee_id NUMBER CONSTRAINT salary_audit_emp_nn NOT NULL,
5     old_salary NUMBER(8,2) CONSTRAINT salary_audit_old_nn NOT NULL,
6     new_salary NUMBER(8,2) CONSTRAINT salary_audit_new_nn NOT NULL,
7     changed_by VARCHAR2(30) CONSTRAINT salary_audit_user_nn NOT NULL,
8     change_date DATE CONSTRAINT salary_audit_date_nn NOT NULL,
9     reason VARCHAR2(200),
10    CONSTRAINT salary_audit_emp_fk FOREIGN KEY (employee_id) REFERENCES employees
11    (employee_id)
12 );
13
14 CREATE OR REPLACE FUNCTION UPDATE_EMP_SALARY_AUDITED(
15     p_emp_id IN NUMBER,
16     p_pct_change IN NUMBER,
17     p_user IN VARCHAR2,
18     p_reason IN VARCHAR2
19 ) RETURN VARCHAR2 IS
20     v_old_sal employees.salary%TYPE;
21     v_new_sal NUMBER(8,2);
22 BEGIN
23     IF p_pct_change > 30 OR p_pct_change < -20 THEN
24         RETURN 'Invalid percentage range';
25     END IF;
```

```
24
25     SELECT salary INTO v_old_sal FROM employees WHERE employee_id = p_emp_id;
26
27     v_new_sal := ROUND(v_old_sal * (1 + p_pct_change / 100), 2);
28
29     UPDATE employees SET salary = v_new_sal WHERE employee_id = p_emp_id;
30
31     INSERT INTO SALARY_AUDIT_LOG (employee_id, old_salary, new_salary, changed_by,
32         change_date, reason)
33     VALUES (p_emp_id, v_old_sal, v_new_sal, p_user, SYSDATE, p_reason);
34
35     COMMIT;
36     RETURN 'Salary updated successfully and audit recorded';
37 EXCEPTION
38     WHEN NO_DATA_FOUND THEN
39         RETURN 'Invalid employee ID';
40     WHEN OTHERS THEN
41         RETURN 'Error: ' || SQLERRM;
42 END UPDATE_EMP_SALARY_AUDITED;
```

7.3 23 – C Online

23-C Question 1: Department Total Salary Limit (\$50,000) Trigger

Write a PL/SQL trigger enforcing: The total salary paid to all employees in any single department must never exceed \$50,000 after any salary modification.

```

1  -- Oracle Compound Trigger avoids Mutating Table Error (ORA-04091)
2  CREATE OR REPLACE TRIGGER trg_cap_dept_salary_50k
3  FOR INSERT OR UPDATE OF salary, department_id ON employees
4  COMPOUND TRIGGER
5
6      TYPE t_dept_set IS TABLE OF BOOLEAN INDEX BY PLS_INTEGER;
7      g_depts t_dept_set;
8
9      AFTER EACH ROW IS
10     BEGIN
11         IF :NEW.department_id IS NOT NULL THEN
12             g_depts(:NEW.department_id) := TRUE;
13         END IF;
14         IF UPDATING AND :OLD.department_id IS NOT NULL THEN
15             g_depts(:OLD.department_id) := TRUE;
16         END IF;
17     END AFTER EACH ROW;
18
19     AFTER STATEMENT IS
20         v_dept_id PLS_INTEGER;
21         v_total NUMBER;
22     BEGIN
23         v_dept_id := g_depts.FIRST;
24         WHILE v_dept_id IS NOT NULL LOOP
25             SELECT NVL(SUM(salary), 0) INTO v_total
26             FROM employees
27             WHERE department_id = v_dept_id;
28
29             IF v_total > 50000 THEN
30                 RAISE_APPLICATION_ERROR(-20013, 'Department ' || v_dept_id || ' total
31                 salary exceeds $50,000 cap (Current: ' || v_total || ')');
32             END IF;
33             v_dept_id := g_depts.NEXT(v_dept_id);
34         END LOOP;
35     END AFTER STATEMENT;
36 END trg_cap_dept_salary_50k;
/

```

23-C Question 2: Employee Overpayment Evaluation Function

Check if employee is paid more than allowed based on tenure and job max salary: - If employee not found, return NULL. - If tenure ≥ 10 yrs: allowed $\leq 95\%$ of max salary. - If tenure $5 \leq \text{yrs} < 10$: allowed $\leq 90\%$ of max salary. - If tenure < 5 yrs: allowed $\leq 85\%$ of max salary. - Return 1 if overpaid, 0 if within allowed range, -1 if job data missing.

```

1  CREATE OR REPLACE FUNCTION CHECK_OVERPAID(p_emp_id IN NUMBER) RETURN NUMBER IS
2      v_sal employees.salary%TYPE;
3      v_hire employees.hire_date%TYPE;
4      v_max_sal jobs.max_salary%TYPE;
5      v_years NUMBER;
6      v_allowed_ratio NUMBER;
7  BEGIN
8      SELECT e.salary, e.hire_date, j.max_salary
9      INTO v_sal, v_hire, v_max_sal
10     FROM employees e

```

```
11     LEFT JOIN jobs j ON e.job_id = j.job_id
12     WHERE e.employee_id = p_emp_id;
13
14     IF v_max_sal IS NULL THEN
15         RETURN -1;
16     END IF;
17
18     v_years := FLOOR(MONTHS_BETWEEN(SYSDATE, v_hire) / 12);
19
20     IF v_years >= 10 THEN
21         v_allowed_ratio := 0.95;
22     ELSIF v_years >= 5 THEN
23         v_allowed_ratio := 0.90;
24     ELSE
25         v_allowed_ratio := 0.85;
26     END IF;
27
28     IF v_sal > (v_allowed_ratio * v_max_sal) THEN
29         RETURN 1;
30     ELSE
31         RETURN 0;
32     END IF;
33 EXCEPTION
34     WHEN NO_DATA_FOUND THEN
35         RETURN NULL;
36 END CHECK_OVERPAID;
37 /
```